Verilog中的表达式位宽

示例一

- (sig1+sig2) 结果应该为“1_1110_0011”，以为计算后的结果应该包含进位位“1”，但是实际result中并没有包含进位位。这是为什么呢？**这是因为表达式中(sig1+sig2)进位操作时会产生一个中间结果，这个中间结果的位宽与sig1和sig2最宽的那一个数保持一致。**

【示例】本示例中，期望将运算结果向右移动一位，这样两个数的和的进位位就可以保留下来。

```
module top_tb;
logic [7:0] sig1,sig2;
logic [7:0] result;

initial begin
    sig1 = 8'hF1;
    sig2 = 8'hF2;
    result = (sig1 + sig2) >>1;
    $display("The sig1 is %0b",sig1);
    $display("The sig2 is %0b",sig2);
    $display("The sig1 + sig2 is %0b",sig1+sig2);
    $display("The result is %0b",result);
end
endmodule
```

硅芯思见

【仿真结果】

```
The sig1 is 11110001
The sig2 is 11110010
The sig1 + sig2 is 11100011
The result is 1110001
Simulation complete, time
```

硅芯思见

加入宽位宽数据拓展结果

- 对sig1和sig2进行加法操作时，增加了一个9位宽的0(可以指定任何一个大于8位的0)，这样子(sig1+sig2+9'b0_0000_0000)进行加法操作时，所有的操作数都被扩展为9位进行计算，(sig1+sig2+9'b0_0000_0000)运算的结果(中间结果)中也将包含进位位，在对这个中间9位宽结果移位时，就会将其进位位一并进行移位

【示例】方法一：加入宽位宽数据拓展结果

```
module top_tb;
logic [7:0] sig1,sig2;
logic [7:0] result;

initial begin
    sig1 = 8'hF1;
    sig2 = 8'hF2;
    result = (sig1 + sig2 + 9'b0_0000_0000) >>1;
    $display("The sig1 is %0b",sig1);
    $display("The sig2 is %0b",sig2);
    $display("The sig1 + sig2 is %0b",sig1+sig2);
    $display("The result is %0b",result);
end
endmodule
```

硅芯思见

【仿真结果】

```
The sig1 is 11110001
The sig2 is 11110010
The sig1 + sig2 is 11100011
The result is 11110001
Simulation complete, time is
```

硅芯思见

使用静态类型转换

- 通过将sig1和sig2中任意一个使用”int'”转换成32位，此时sig1和sig2进行操作时，所有的操作数都会拓展为32位后进行计算

【示例】方法二：使用静态类型转换

```
module top_tb;
logic [7:0] sig1,sig2;
logic [7:0] result;

initial begin
    sig1 = 8'hF1;
    sig2 = 8'hF2;
    result = (int'(sig1) + sig2) >>1;
    $display("The sig1 is %0b",sig1);
    $display("The sig2 is %0b",sig2);
    $display("The sig1 + sig2 is %0b",sig1+sig2);
    $display("The result is %0b",result);
end
endmodule
```

硅芯思见

扩展赋值表达式中相关操作数的位宽

- 将**result**的位宽声明为**9**位宽，那么当**sig1**和**sig2**进行运算时，会被扩展为**9**位宽，此时**sig1+sig2**的结果也为**9**位，即此时两个数相加结果的进位位被保留下来，所以此时进行移位操作后输出结果包含进位位。

【示例】方法三：扩展赋值表达式中相关操作数的位宽

```
1  module top_tb;
2  logic [7:0] sig1,sig2;
3  logic [8:0] result;
4
5  initial begin
6      sig1 = 8'hF1;
7      sig2 = 8'hF2;
8      result = (sig1 + sig2) >>1;
9      $display("The sig1 is %0b",sig1);
10     $display("The sig2 is %0b",sig2);
11     $display("The sig1 + sig2 is %0b",sig1+sig2);
12     $display("The result is %0b",result);
13 end
14 endmodule
```

硅芯思见

Verilog和SystemVerilog表达式解析规则

- 表达式分为了两类：self-determined和context-determined
  - self-determined表达式的位宽仅由表达式中操作数的位宽决定，这样的表达式的位宽不会受到表达式上下文的影响，也就是不会受到表达式其他部分的影响
  - context-determined表达式的位宽既由表达式决定，也由整个语句的其他部分表达式位宽决定

Expression	Bit Length	Comments
Assignment statement = <=	context	Both sides of assignment affect size extension.
Assignment operations += -= *= / % &= |= ^=	context	Both sides of assignment affect size extension. Left operand is part of the right-hand side assignment context. (e.g. a+=b expands to a = a+b)
Unsized constant number	Same as integer	
Sized constant number	As given	
i op j, where op is: + - * / % & | ^ ^~ ~^	max(L(i),L(j))	
op i, where op is: + - ~	L(i)	
i op j, where op is: === !== == != > >= < <=	1 bit	Operands are sized to max(L(i), L(j))
i op j, where op is: && || -> <->	1 bit	All operands are self-determined
op i, where op is: & ~& | ~| ^ ~^ ^~ !	1 bit	All operands are self-determined
i op j, where op is: >> << ** >>> <<<	L(i)	j is self-determined
i ? j : k	max(L(j),L(k))	i is self-determined
{i,...,j}	L(i)+...+L(j)	All operands are self-determined
{i{j,...,k}}	ix(L(j)+...+L(k))	All operands are self-determined

硅芯思见

示例一

- &sig1和&sig2并未出现在赋值表达式中，并且符合“& op”条件，所有操作数位宽self-determined，通过上表分析&sig1和&sig2的结果均为1位
- &sig1和&sig2并未出现在赋值表达式中，&sig1和&sig2分别计算完成后的结果均为1位，并且这两个&运算的结果位于“+”左右两侧，两个1位数相加的结果仍为1位，所以仿真结果为0
- “&sig1+sig2”和“sig1+&sig2”并未出现在赋值表达式中，所以所有操作数的位宽self-determined，sig1和sig2均位于“+”运算符两侧，此时“+”操作结果位宽取决于“+”操作符操作数最大位宽，所以在4th中显示的结果与sig1的位宽相同，而5th中显示的结果与sig2位宽相同
- 在“result =（&sig1 + &sig2）>>1”中，因为&sig1和&sig2符合“& op”条件，所以先会计算&sig1和&sig2的结果，应该为1位，但是 **作为赋值表达式(赋值表达式右侧操作数位宽由context-determined决定，即由赋值表达式左右两侧位宽最宽的数决定)右侧的操作数“&sig1”和“&sig2”的位宽由赋值表达式两侧的操作数共同决定，因为表达式左侧的result位宽为10位，所以&sig1和&sig2进行“+”操作时，位宽会被扩展为10位宽**
- 在“result = (sig1 + sig2) >>1”中，sig1和sig2在进行计算时的位宽由赋值表达式两侧的操作数共同决定，因为表达式左侧的result位宽为10位，所以sig1和sig2进行“+”操作时，位宽会被扩展为10位宽

```
module top_tb;
logic [7:0] sig1;
logic [8:0] sig2;
logic [9:0] result;

initial begin
    sig1 = 8'hFF;
    sig2 = 9'h1FF;
    $display("1st : The &sig1 is %b",&sig1);
    $display("2nd : The &sig2 is %b",&sig2);
    $display("3rd : The &sig1+&sig2 is %b",&sig1+&sig2);
    $display("4th : The sig1+&sig2 is %b",sig1+&sig2);
    $display("5th : The &sig1+sig2 is %b",&sig1+sig2);
    $display("6th : The sig1+sig2 is %b",sig1+sig2);
    $display("================================================");
    result = (&sig1 + &sig2) >> 1;
    $display("7th : The (&sig1+&sig2)>>1 result is %b",result);
    result = (sig1 + &sig2) >> 1;
    $display("8th : The (sig1+&sig2)>>1 result is %b",result);
    result = (&sig1 + sig2) >> 1;
    $display("9th : The (&sig1+sig2)>>1 result is %b",result);
    result = (sig1 + sig2) >> 1;
    $display("10th : The (sig1+sig2)>>1 result is %b",result);
    $display("================================================");
    result = (&sig1 + &sig2);
    $display("11th : The (&sig1+&sig2) result is %b",result);
    result = (sig1 + &sig2);
    $display("12th : The (sig1+&sig2) result is %b",result);
    result = (&sig1 + sig2);
    $display("13th : The (&sig1+sig2) result is %b",result);
    result = (sig1 + sig2);
    $display("14th : The (sig1+sig2) result is %b",result);
end
endmodule
```

硅芯思见

```
1st : The &sig1 is 1
2nd : The &sig2 is 1
3rd : The &sig1+&sig2 is 0
4th : The sig1+&sig2 is 00000000
5th : The &sig1+sig2 is 000000000
6th : The sig1+sig2 is 011111110
================================================
7th : The (&sig1+&sig2)>>1 result is 0000000001
8th : The (sig1+&sig2)>>1 result is 0010000000
9th : The (&sig1+sig2)>>1 result is 0100000000
10th : The (sig1+sig2)>>1 result is 0101111111
================================================
11th : The (&sig1+&sig2) result is 0000000010
12th : The (sig1+&sig2) result is 0100000000
13th : The (&sig1+sig2) result is 1000000000
14th : The (sig1+sig2) result is 1011111110
Simulation complete, time is 0.
```

硅芯思见

示例二

- 当操作数位于赋值表达式左右两侧时，赋值表达式最终结果的位宽由赋值表达式赋值表达式左侧操作数决定，但是其中的各种运算及中间结果位宽需要对赋值表达式两侧数的位宽都进行考虑，即context-determined。

```
module top_tb;
logic [7:0] sig1;
logic [8:0] sig2;
logic [6:0] result;

initial begin
    sig1 = 8'hFF;
    sig2 = 9'h1FF;
    $display("1st : The &sig1 is %b",&sig1);
    $display("2nd : The &sig2 is %b",&sig2);
    $display("3rd : The &sig1+&sig2 is %b",&sig1+&sig2);
    $display("4th : The sig1+&sig2 is %b",sig1+&sig2);
    $display("5th : The &sig1+sig2 is %b",&sig1+sig2);
    $display("6th : The sig1+sig2 is %b",sig1+sig2);
    $display("================================================");
    result = (&sig1 + &sig2) >> 1;
    $display("7th : The (&sig1+&sig2)>>1 result is %b",result);
    result = (sig1 + &sig2) >> 1;
    $display("8th : The (sig1+&sig2)>>1 result is %b",result);
    result = (&sig1 + sig2) >> 1;
    $display("9th : The (&sig1+sig2)>>1 result is %b",result);
    result = (sig1 + sig2) >> 1;
    $display("10th : The (sig1+sig2)>>1 result is %b",result);
    $display("------------------------------------------------");
    result = (&sig1 + &sig2);
    $display("11th : The (&sig1+&sig2) result is %b",result);
    result = (sig1 + &sig2);
    $display("12th : The (sig1+&sig2) result is %b",result);
    result = (&sig1 + sig2);
    $display("13th : The (&sig1+sig2) result is %b",result);
    result = (sig1 + sig2);
    $display("14th : The (sig1+sig2) result is %b",result);
end
endmodule
```

硅芯思见

```
1st : The &sig1 is 1
2nd : The &sig2 is 1
3rd : The &sig1+&sig2 is 0
4th : The sig1+&sig2 is 00000000
5th : The &sig1+sig2 is 000000000
6th : The sig1+sig2 is 011111110
================================================
7th : The (&sig1+&sig2)>>1 result is 0000001
8th : The (sig1+&sig2)>>1 result is 0000000
9th : The (&sig1+sig2)>>1 result is 0000000
10th : The (sig1+sig2)>>1 result is 1111111
================================================
11th : The (&sig1+&sig2) result is 0000010
12th : The (sig1+&sig2) result is 0000000
13th : The (&sig1+sig2) result is 0000000
14th : The (sig1+sig2) result is 1111110
Simulation complete, time is 0.
```

硅芯思见

- 为了获得期望的结果，在使用各种操作数进行运算时，需要注意以下几点：
    - 操作数的符号特性，是否为有符号数，因为牵扯到数据拓宽时高位补位
    - 操作数的位宽，特别是在赋值表达式中位于赋值表达式两侧操作数的位宽
    - 操作表达式所使用的运算符（self-determined）

- 表达式是否位于赋值表达式中，决定了是self-determined还是context-determined

总结

- 第一步：先确定赋值表达式左右两侧的self-determined运算，并对其完成解析；
- 第二步：根据赋值表达式两侧最大的位宽对赋值表达式左右两侧表达式的位宽进行扩展；
- 第三步：左右两侧拓展后的操作数进行相关运算，但是需要注意，赋值表达式的最终结果将不会被拓展；